

Module 8 Exercises

Reusable Business Logic

Environment	Yellowbrick Cloud Test Environment
Tools	Yellowbrick Manager (SQL Editor) or DBeaver
Sample data	samples.retail — accessible to all students
Your database	db_<username> (e.g. db_teststudent)
Schema prefix	Always use samples.retail.<table> for source data

Part	What You Will Do	Time
Part 1	Create and query Views — simplify joins and encode business logic	~15 min
Part 2	Write VOID Stored Procedures — CALL with RAISE INFO feedback	~15 min
Part 3	Write SETOF Stored Procedures — return table-shaped results	~20 min
Part 4	SAS to SQL — translate PROC MEANS, PROC FREQ, and fix row loops	~15 min
Part 5 (Stretch)	Python — connect with psycopg2 and analyse with pandas	~15 min

Part 1 — Views

A view is a named, saved SELECT query. No data is stored — Yellowbrick re-executes the query every time you reference it. In this part you will create two views against **samples.retail** and query them like regular tables.

Time: ~15 minutes

1.1 Create vw_sales_detail

The retail schema has five tables: **fact_sales**, **dim_date**, **dim_store**, **dim_customer**, **dim_product**. Joining them every time is repetitive. We will encode the join once as a view.

Run it

Create the view. This encodes the 5-table join in one place so any analyst can query it without knowing the underlying structure.

```
CREATE OR REPLACE VIEW public.vw_sales_detail AS
SELECT
    f.transaction_id,
    d.full_date           AS sale_date,
    d.month_name, d.year, d.quarter, d.is_weekend,
```

```

s.store_name, s.city, s.state,
c.first_name || ' ' || c.last_name AS customer_name,
c.membership_tier,
p.name AS product_name,
p.category, p.brand,
f.quantity, f.unit_price, f.discount_amount, f.total,
f.payment_status
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_date d ON f.date_key = d.date_key
JOIN samples.retail.dim_store s ON f.store_key = s.store_key
JOIN samples.retail.dim_customer c ON f.customer_key = c.customer_key
JOIN samples.retail.dim_product p ON f.product_key = p.product_key;

```

Now query the view like a table — no JOIN required:

```

SELECT category
,      ROUND(SUM(total), 2) AS revenue
FROM   public.vw_sales_detail
WHERE  year = 2026
GROUP BY category
ORDER BY revenue DESC;

```

Expected: 20 rows. Furniture leads with ~\$165M, Clothing second (~\$164M).

Modify it

Add a filter for Electronics stores only. Show **store_name** and **revenue**. Which store sold the most Electronics in 2026?

```

SELECT store_name
,      ROUND(SUM(total), 2) AS revenue
FROM   public.vw_sales_detail
WHERE  year = 2026
      AND category = 'Electronics'
GROUP BY store_name
ORDER BY revenue DESC
LIMIT 5;

```

Expected: Super Retail Store #14 leads with ~\$1.86M in Electronics revenue.

Your turn

Using **vw_sales_detail**, write a query that shows total revenue and transaction count for **Gold** and **Platinum** members only, grouped by **membership_tier**.

Hint: use `COUNT(DISTINCT transaction_id)` for transaction count.

1.2 Create vw_customer_spend

This view pre-aggregates customer metrics so analysts never have to write GROUP BY boilerplate.

 Run it

Create the aggregated customer view:

```

CREATE OR REPLACE VIEW public.vw_customer_spend AS
SELECT
  c.customer_id,
  c.first_name || ' ' || c.last_name AS customer_name,
  c.membership_tier,
  COUNT(DISTINCT f.transaction_id) AS transaction_count,
  ROUND(SUM(f.total), 2) AS total_spend,
  ROUND(AVG(f.total), 2) AS avg_basket_size
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_customer c ON f.customer_key = c.customer_key
GROUP BY 1, 2, 3;

```

Query it for the top 5 Gold customers:

```

SELECT customer_name, membership_tier, total_spend, transaction_count
FROM public.vw_customer_spend
WHERE membership_tier = 'Gold'
ORDER BY total_spend DESC
LIMIT 5;

```

Expected: Jessica Brown leads Gold tier with ~\$230K across 20 visits.

 Modify it

Change 'Gold' to 'Platinum'. Who are the top 5 Platinum spenders?

```

SELECT customer_name, membership_tier, total_spend, transaction_count
FROM public.vw_customer_spend
WHERE membership_tier = 'Platinum'
ORDER BY total_spend DESC
LIMIT 5;

```

 Your turn

Using **vw_customer_spend**, find customers who have visited more than 15 times (`transaction_count > 15`).

Show **customer_name**, **membership_tier**, **transaction_count**, and **avg_basket_size**. Order by **transaction_count** descending. Limit to 10.

 Tip

Views are 'late-bound' in Yellowbrick — table references resolve at query time. This means a view created in your database can reference tables in **samples.retail** without issue. If the underlying tables change columns, DROP and recreate the view.

Part 2 — Stored Procedures: VOID

A VOID procedure performs actions but returns no rows. You call it with `CALL` and use `RAISE INFO` to print output to your SQL client. VOID procedures are ideal for ETL steps, data quality checks, and summary logging.

Time: ~15 minutes

2.1 Create and CALL show_category_summary

Run it

Create the procedure:

```
CREATE OR REPLACE PROCEDURE
  public.show_category_summary(p_category VARCHAR(50))
  RETURNS VOID
  LANGUAGE plpgsql
AS $$
DECLARE
  v_items   BIGINT;
  v_units   BIGINT;
  v_revenue NUMERIC(15,2);
BEGIN
  SELECT COUNT(*)
    ,      SUM(f.quantity)
    ,      ROUND(SUM(f.total), 2)
  INTO   v_items, v_units, v_revenue
  FROM   samples.retail.fact_sales f
  JOIN   samples.retail.dim_product p
    ON   f.product_key = p.product_key
  WHERE  p.category = p_category;

  RAISE INFO 'Category:   %', p_category;
  RAISE INFO 'Line Items: %', v_items;
  RAISE INFO 'Units Sold: %', v_units;
  RAISE INFO 'Revenue:    $%', v_revenue;
END;
$;
```

Call it with Electronics:

```
CALL public.show_category_summary('Electronics');
```

Expected: RAISE INFO output showing 274,606 line items, 824,000 units sold, \$442M revenue.

Modify it

Call the same procedure with 'Furniture'. Which category has higher revenue?

```
CALL public.show_category_summary('Furniture');
CALL public.show_category_summary('Clothing');
```

Expected: Furniture leads all categories with ~\$447M. Clothing is second at ~\$443M.

2.2 Write your own VOID procedure

Your turn

Create a procedure called **show_store_summary(p_city VARCHAR(50))** that reports: number of distinct stores, number of distinct transactions, and total revenue for all stores in a given city.

Use `RAISE INFO` to print all three values, labelled clearly.

Test it: `CALL public.show_store_summary('Oakland');`

Expected: City: Oakland | Stores: 1 | Transactions: 9947 | Revenue: ~\$16.1M

Tip

Key rules for VOID procedures:

- Call with `CALL` — not `SELECT`
- The SQL inside the procedure runs on the compute workers (fast)
- The PL/pgSQL logic (`DECLARE`, `BEGIN`, `RAISE INFO`) runs on the manager node
- Yellowbrick only supports `LANGUAGE plpgsql` — no SQL, Python, or other languages

Part 3 — Stored Procedures: SETOF

A SETOF procedure returns rows shaped like an existing table. You call it with `SELECT * FROM proc()` — not `CALL`. This lets downstream queries filter, sort, or join the result just like a regular table.

Time: ~20 minutes

3.1 Create get_top_customers

Run it

Step 1: Create the result-type table. The procedure's return shape must match this table exactly.

```
CREATE TABLE IF NOT EXISTS public.top_customer_result (
  customer_name  VARCHAR(100),
  membership_tier VARCHAR(20),
  total_spend    NUMERIC(15,2),
  visits         BIGINT
) DISTRIBUTE ON (customer_name);
```

Step 2: Create the procedure.

```
CREATE OR REPLACE PROCEDURE public.get_top_customers(p_limit INT)
  RETURNS SETOF public.top_customer_result
```

```

LANGUAGE plpgsql
AS $$
DECLARE
    v_row public.top_customer_result%ROWTYPE;
    v_sql VARCHAR(1000) = '
        SELECT
            c.first_name || ' ' || c.last_name,
            c.membership_tier,
            ROUND(SUM(f.total), 2),
            COUNT(DISTINCT f.transaction_id)
        FROM samples.retail.fact_sales f
        JOIN samples.retail.dim_customer c
          ON f.customer_key = c.customer_key
        GROUP BY 1, 2
        ORDER BY 3 DESC
        LIMIT $1';
BEGIN
    FOR v_row IN EXECUTE(v_sql) USING p_limit
    LOOP
        RETURN NEXT v_row;
    END LOOP;
END;
$$;

```

Step 3: Call with SELECT * FROM (not CALL!)

```
SELECT * FROM public.get_top_customers(5::INT);
```

Expected: Top 5 customers across all tiers. Bronze members dominate with 1100+ visits and ~\$10M total spend each.

 Modify it

Change the limit to 10. Then add a WHERE clause to filter to 'Platinum' tier only. What is the difference between filtering inside the proc vs. outside?

```

-- Filter OUTSIDE the proc (runs on manager -- less efficient for large results):
SELECT * FROM public.get_top_customers(100::INT)
WHERE membership_tier = 'Platinum'
ORDER BY total_spend DESC
LIMIT 5;

```

Expected: Top Platinum customers shown. Note: filtering outside means 100 rows are materialised on the manager before the WHERE is applied.

3.2 Add an overloaded version with a tier filter
 Run it

Create a second version with the same name but a different signature. Yellowbrick routes to the matching version automatically.

```

CREATE OR REPLACE PROCEDURE
  public.get_top_customers(p_limit INT, p_tier VARCHAR(20))
  RETURNS SETOF public.top_customer_result
  LANGUAGE plpgsql
AS $$
DECLARE
  v_row public.top_customer_result%ROWTYPE;
  v_sql VARCHAR(1000) = '
    SELECT
      c.first_name || ' ' || c.last_name,
      c.membership_tier,
      ROUND(SUM(f.total), 2),
      COUNT(DISTINCT f.transaction_id)
    FROM samples.retail.fact_sales f
    JOIN samples.retail.dim_customer c
      ON f.customer_key = c.customer_key
    WHERE c.membership_tier = $2
    GROUP BY 1, 2
    ORDER BY 3 DESC
    LIMIT $1';
BEGIN
  FOR v_row IN EXECUTE(v_sql) USING p_limit, p_tier
  LOOP
    RETURN NEXT v_row;
  END LOOP;
END;
$$;

-- The tier filter happens INSIDE the SQL -- runs on the workers
SELECT * FROM public.get_top_customers(5::INT, 'Platinum');

```

Expected: Top 5 Platinum customers. Elizabeth Taylor leads with ~\$2.0M spend.

Modify it

Call the overloaded version for 'Gold' tier. Then compare: which tier has higher avg spend per visit (total_spend / visits)?

```

SELECT customer_name, membership_tier, total_spend, visits,
       ROUND(total_spend / visits, 2) AS avg_per_visit
FROM   public.get_top_customers(10::INT, 'Gold')
ORDER BY avg_per_visit DESC;

```

3.3 Write get_top_products

Your turn

Create a result-type table **public.top_product_result** with columns: `product_name VARCHAR(200)`, `category VARCHAR(50)`, `units_sold BIGINT`, `revenue NUMERIC(15,2)`.

Then create a procedure **get_top_products(p_limit INT)** that returns the top N products by revenue, joining **fact_sales** to **dim_product**.

Test it: `SELECT * FROM public.get_top_products(5::INT);`

Expected: Top product is a Toys & Games item with ~\$481K revenue.

Stretch: Add a `p_category VARCHAR(50)` parameter to the overloaded version so you can call `get_top_products(5::INT, 'Electronics')`.

Part 4 — SAS to Yellowbrick SQL

The most important shift when migrating from SAS to Yellowbrick is thinking in **sets**, not **rows**. This part maps common SAS patterns to their SQL equivalents.

Time: ~15 minutes

4.1 PROC MEANS → GROUP BY aggregation

In SAS, `PROC MEANS` computes summary statistics by group. In SQL, this is a `SELECT` with `GROUP BY` and aggregate functions.

Run it

This replaces: `proc means data=sales sum mean max min; class category; var total; run;`

```
SELECT
  p.category,
  COUNT(*)           AS line_items,
  ROUND(SUM(f.total), 2) AS total_revenue,
  ROUND(AVG(f.total), 2) AS avg_sale,
  ROUND(MAX(f.total), 2) AS max_sale,
  ROUND(MIN(f.total), 2) AS min_sale
FROM  samples.retail.fact_sales f
JOIN  samples.retail.dim_product p ON f.product_key = p.product_key
GROUP BY p.category
ORDER BY total_revenue DESC;
```

Expected: 20 rows. Furniture leads at ~\$447M. All categories have similar max sales (~\$5,400).

Modify it

Add a `HAVING` clause to show only categories with total revenue over \$440M. How many categories qualify?

```
SELECT
  p.category,
  ROUND(SUM(f.total), 2) AS total_revenue,
  ROUND(AVG(f.total), 2) AS avg_sale
FROM  samples.retail.fact_sales f
JOIN  samples.retail.dim_product p ON f.product_key = p.product_key
GROUP BY p.category
HAVING ROUND(SUM(f.total), 2) > 440000000
ORDER BY total_revenue DESC;
```


Expected: 3 categories: Furniture, Clothing, Electronics.

Your turn

Write a PROC MEANS equivalent that reports **total revenue and transaction count per store** (join **fact_sales** to **dim_store**). Which store has the highest total revenue overall?

4.2 PROC FREQ → COUNT with percentage

In SAS, PROC FREQ produces a frequency table with counts and percentages. In SQL, use COUNT with a window function for the percentage calculation.

Run it

This replaces: `proc freq data=transactions; tables membership_tier; run;`

```
SELECT
    c.membership_tier,
    COUNT(DISTINCT f.transaction_id)                                AS frequency,
    ROUND(100.0 * COUNT(DISTINCT f.transaction_id) /
          SUM(COUNT(DISTINCT f.transaction_id)) OVER (), 1)        AS pct
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_customer c ON f.customer_key = c.customer_key
GROUP BY 1
ORDER BY frequency DESC;
```

Expected: Bronze 49.7% | Silver 30.2% | Gold 15.1% | Platinum 5.0%

Modify it

Change the grouping to **product category** instead of membership tier. Join **dim_product** to get **p.category**. Are all categories roughly equal in transaction frequency?

```
SELECT
    p.category,
    COUNT(DISTINCT f.transaction_id)                                AS frequency,
    ROUND(100.0 * COUNT(DISTINCT f.transaction_id) /
          SUM(COUNT(DISTINCT f.transaction_id)) OVER (), 1)        AS pct
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p ON f.product_key = p.product_key
GROUP BY 1
ORDER BY frequency DESC;
```

Your turn

Write a query that shows **weekend vs. weekday** transactions and revenue. Join **dim_date** and use **d.is_weekend**.

Show: `day_type` (Weekend/Weekday), `transactions`, `revenue`, and `avg_sale`.

Expected: Weekdays account for ~71% of transactions, similar avg sale to weekends (~\$1,584).

4.3 The Cursor Loop Anti-Pattern

The most common SAS migration mistake: replacing a DATA step iteration with a cursor loop in SQL. A cursor fires one SQL statement per row and saturates the manager node. Always use a single set-based statement instead.

Run it

Setup: Create a staging table to work with.

```
DROP TABLE IF EXISTS public.sales_staging;
CREATE TABLE public.sales_staging AS
SELECT
    f.transaction_id,
    f.total,
    'pending'::VARCHAR(20) AS review_status
FROM samples.retail.fact_sales f
WHERE f.total > 4000
LIMIT 100;

-- Check initial state
SELECT review_status, COUNT(*) AS cnt
FROM public.sales_staging
GROUP BY review_status;
```

Expected: 100 rows, all with review_status = 'pending'.

The RIGHT way: One set-based UPDATE replaces any cursor loop.

```
UPDATE public.sales_staging
SET    review_status = 'high_value'
WHERE  total > 4500;

SELECT review_status, COUNT(*) AS cnt
FROM public.sales_staging
GROUP BY review_status;
```

Expected: ~51 rows flagged 'high_value', ~49 still 'pending'. One statement updated all matching rows simultaneously.

Modify it

Reset all rows to 'pending', then write a single UPDATE that categorises rows into **three** groups using CASE WHEN:

```
-- Reset
UPDATE public.sales_staging SET review_status = 'pending';

-- Flag three categories in one statement
```

```

UPDATE public.sales_staging
SET    review_status = CASE
        WHEN total > 4500 THEN 'high_value'
        WHEN total BETWEEN 4000 AND 4500 THEN 'medium_value'
        ELSE 'pending'
      END;

SELECT review_status, COUNT(*) AS cnt
FROM   public.sales_staging
GROUP BY review_status;

```

Your turn

Reset all rows to 'pending', then write a single set-based UPDATE that sets `review_status` = 'completed' for all rows where `total > 4200`.

Verify with `SELECT COUNT(*) WHERE review_status = 'completed'`. How many rows were updated?

Part 5 (Stretch) — Python with psycopg2

These exercises use Python in a Jupyter notebook. Open Jupyter, create a new notebook, and work through each cell. You will need your Yellowbrick host, username, and password.

Time: ~15 minutes

5.1 Connect to Yellowbrick

Run it

Install and import the required libraries, then establish a connection:

```

# Cell 1: Install if needed (run once)
# !pip install psycopg2-binary pandas matplotlib

import psycopg2
import pandas as pd
import matplotlib.pyplot as plt

# Cell 2: Connect -- replace with your credentials
conn = psycopg2.connect(
    host      = 'YOUR_YB_HOST',
    port     = 5432,
    dbname    = 'samples',
    user      = 'YOUR_USERNAME',
    password  = 'YOUR_PASSWORD',
    sslmode   = 'require'
)

# Test the connection

```

```
df_test = pd.read_sql('SELECT current_database(), current_user', conn)
print(df_test)
```

Expected: One row showing database='samples' and your username.

5.2 PROC MEANS in Python

Run it

Pull aggregated revenue by category into a DataFrame and run `describe()` on it:

```
sql = '''
SELECT
    p.category,
    ROUND(SUM(f.total), 2)          AS revenue,
    COUNT(DISTINCT f.transaction_id) AS transactions
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p ON f.product_key = p.product_key
GROUP BY p.category
ORDER BY revenue DESC
'''

df = pd.read_sql(sql, conn)

# PROC MEANS equivalent
print(df['revenue'].describe())
print()
print(df.head(5))
```

Expected: 20 rows. `describe()` shows count=20, mean=~\$434M, max=~\$447M (Furniture).

Modify it

Filter the DataFrame in Python to show only categories with revenue > \$440M. Then rename the columns with `df.rename()` and export to Excel:

```
# Filter like SAS WHERE
top = df[df['revenue'] > 440_000_000]
print(top)

# Rename columns (like SAS RENAME=)
top = top.rename(columns={'category': 'Category', 'revenue': 'Revenue ($)'})

# Export to Excel (like ODS Excel)
top.to_excel('top_categories.xlsx', index=False)
print('Exported!')
```

5.3 PROC GCHART in Python

 Your turn

Pull monthly revenue for 2026 from Yellowbrick, load it into a DataFrame, and create a bar chart showing revenue per month.

SQL to use in `pd.read_sql()`:

```
SELECT
    d.month_number,
    d.month_name,
    ROUND(SUM(f.total), 2) AS revenue
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_date d ON f.date_key = d.date_key
WHERE d.year = 2026
GROUP BY 1, 2
ORDER BY 1;
```

Then create the chart with matplotlib (like `PROC GCHART`):

```
df_monthly = pd.read_sql(sql, conn)

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(df_monthly['month_name'], df_monthly['revenue'] / 1e6, color='#0D8FA4')
ax.set_xlabel('Month')
ax.set_ylabel('Revenue ($M)')
ax.set_title('Monthly Revenue - 2026')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

conn.close()
```

Expected: A bar chart with 5 monthly bars. Jan and Mar are the tallest months.